

Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

College of Computer Studies

ITEC 204 - Data Structures and Algorithms
Laboratory Activity 1 - Documentation

Name:	Zenrik Principe
Section:	BSIT-2A
Date:	September 24, 2026
Instructor:	Maxyne Nuela Ignacio

I. Program Title

IT Automation Incident Ticket Manager

II. Objective

This program is designed to simulate an IT Automation Incident Ticket Manager, which helps support teams organize and manage incident tickets generated by automation bots. It applies the concepts of a linear data structure, specifically a list of dictionaries, to store and manipulate ticket records. Through this program, basic data structure operations such as adding, displaying, searching, removing, and counting records are demonstrated in a practical, menu-driven application.

III. Scenario

Automation bots running in an IT environment (such as BOT-Inventory, BOT-Email, and BOT-DataSync) occasionally encounter errors or failures during their scheduled tasks. When this happens, an incident ticket is automatically generated and needs to be tracked by the support team until it is resolved. This program simulates the ticket-tracking system used by the support team to add new incidents, view active tickets, search for specific tickets, remove resolved ones, and monitor how many incidents are still active.

IV. Data Structure Used

A List of Dictionaries was used as the linear data structure for this program. Each incident ticket is represented as a dictionary containing three key-value pairs: id (Incident ID), bot (the automation bot involved), and description (short description of the issue). All ticket dictionaries are stored inside a single list named `incident_tickets`. A list was chosen because it naturally supports sequential (linear) storage of records and provides built-in operations such as `append()` and `remove()` that map directly to the required Add and Remove operations, while allowing simple iteration for Display, Search, and Count.

V. Program Operations

Add Incident - Appends a new ticket (as a dictionary) to the end of the list.

Code: `incident_tickets.append(ticket)`

When the user chooses to add a ticket, the program collects the Incident ID, Bot name, and Short Description, packages them into a dictionary, and appends this dictionary to the `incident_tickets` list using Python's built-in `append()` method.

Display Incidents - Iterates through the list and prints each ticket's details.

Code: `for ticket in incident_tickets:`

The program loops through every dictionary currently stored in `incident_tickets` and prints the Incident ID, Bot, and Description of each one in a formatted table, allowing the user to view all active tickets at once.

Search Incident - Performs a linear search for a matching Incident ID.

Code: `if ticket["id"] == incident_id:`

The program checks each ticket in the list one by one, comparing its `id` field against the Incident ID entered by the user. Once a match is found, the ticket's details are displayed; if no match is found after checking the entire list, a not-found message is shown.

Remove Resolved Incident - Deletes a specific ticket dictionary from the list.

Code: `incident_tickets.remove(ticket)`

After locating the ticket that matches the given Incident ID (using the same linear search logic), the program calls `remove()` to delete that specific dictionary from the `incident_tickets` list, effectively marking the incident as resolved and removing it from the active list.

Count Active Incidents - Returns the current number of tickets in the list.

Code: `len(incident_tickets)`

The program uses Python's built-in `len()` function to count how many dictionaries currently exist in the `incident_tickets` list, giving the total number of active (unresolved) incident tickets at any point in time.

Flowchart

The flowchart below illustrates the overall process and logic of the program, from initialization and loading of sample data, through the menu-driven loop, down to performing the selected operation and looping back until the user exits.

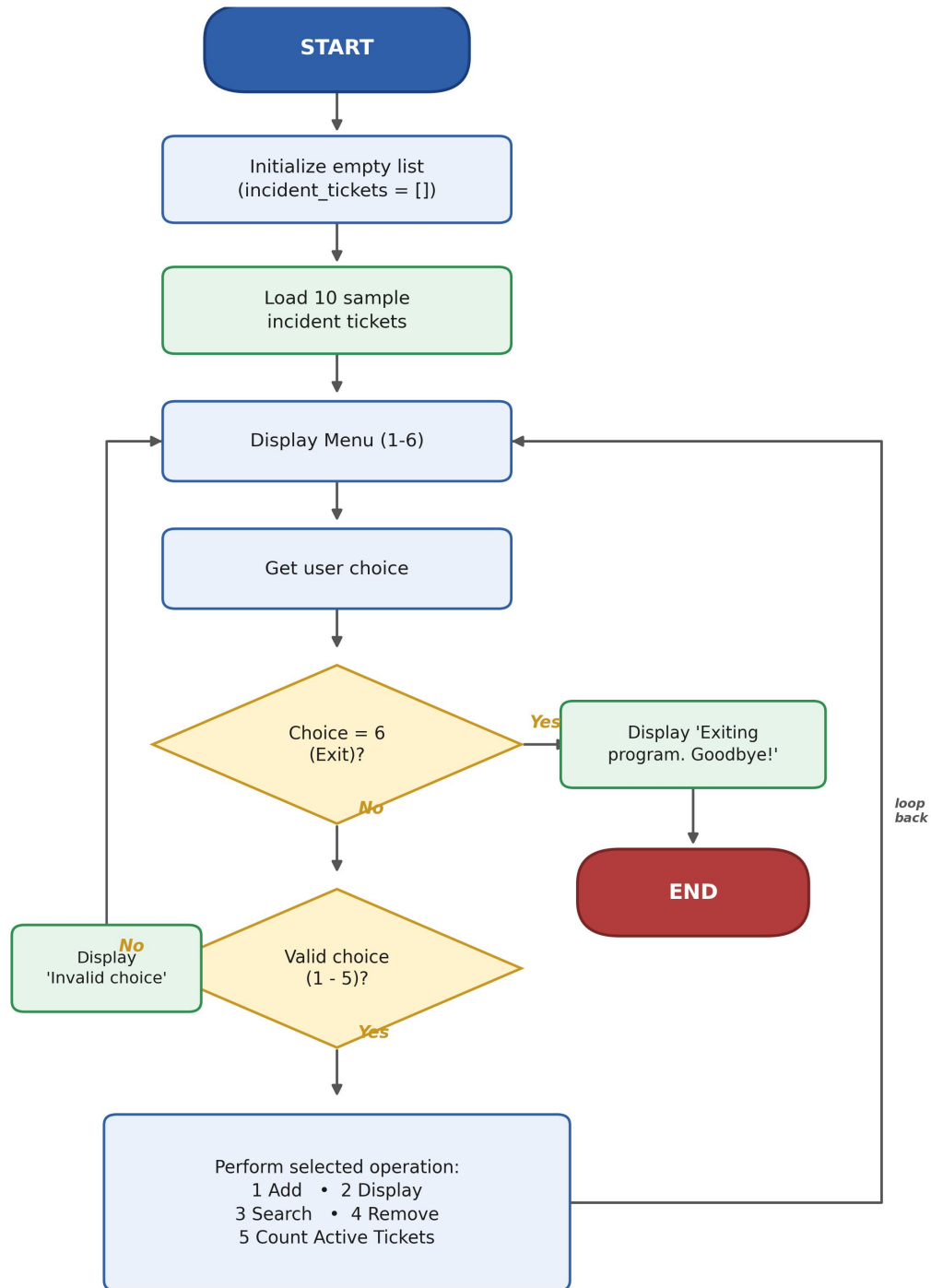


Figure 0. Overall process and logic of the IT Automation Incident Ticket Manager.

VI. Source Code

"""

IT Automation Incident Ticket Manager
ITEC 204 - Data Structures and Algorithms
Laboratory Activity 1

Linear Data Structure Used: List of Dictionaries
"""

```
incident_tickets = [] # main linear data structure (list)
```

```
def add_ticket(incident_id, bot, description):  
    """Add a new incident ticket to the list."""  
    ticket = {"id": incident_id, "bot": bot, "description": description}  
    incident_tickets.append(ticket)  
    print(f"\nTicket {incident_id} added successfully.")
```

```
def display_tickets():  
    """Display all active incident tickets."""  
    if not incident_tickets:  
        print("\nNo active incident tickets.")  
        return  
    print("\n" + "-" * 75)  
    print(f"{'Incident ID':<15}{'Bot':<20}{'Short Description'}")  
    print("-" * 75)  
    for ticket in incident_tickets:  
        print(f"{ticket['id']:<15}{ticket['bot']:<20}{ticket['description']}")  
    print("-" * 75)
```

```
def search_ticket(incident_id):  
    """Search for a specific incident ticket using its Incident ID."""  
    for ticket in incident_tickets:  
        if ticket["id"] == incident_id:  
            print("\nTicket Found:")  
            print(f"Incident ID : {ticket['id']}")  
            print(f"Bot          : {ticket['bot']}")  
            print(f"Description : {ticket['description']}")  
            return ticket  
    print(f"\nTicket {incident_id} not found.")  
    return None
```

```
def remove_ticket(incident_id):  
    """Remove a resolved incident ticket from the list."""  
    for ticket in incident_tickets:  
        if ticket["id"] == incident_id:  
            incident_tickets.remove(ticket)  
            print(f"\nTicket {incident_id} has been marked resolved and removed.")  
            return  
    print(f"\nTicket {incident_id} not found.")
```

```
def count_active_tickets():  
    """Display the total number of active incident tickets."""  
    print(f"\nTotal active incident tickets: {len(incident_tickets)}")
```

```
def load_sample_data():  
    """Pre-load the 10 sample incident tickets for testing."""  
    sample_data = [
```

```

        ("INC1392939", "BOT-Inventory", "Failed to generate the daily report"),
        ("INC1392940", "BOT-Email", "Failed to send the scheduled notification"),
        ("INC1392941", "BOT-DataSync", "Encountered an error during data transfer"),
        ("INC1392942", "BOT-Invoice", "Failed to process an invoice"),
        ("INC1392943", "BOT-Report", "Failed to generate the weekly report"),
        ("INC1392944", "BOT-FileTransfer", "Failed to upload the required file"),
        ("INC1392945", "BOT-DataEntry", "Encountered an error while entering records"),
        ("INC1392946", "BOT-Backup", "Failed to complete the scheduled backup"),
        ("INC1392947", "BOT-Validation", "Failed to validate the submitted records"),
        ("INC1392948", "BOT-Notification", "Failed to send the system alert"),
    ]
    for incident_id, bot, description in sample_data:
        add_ticket(incident_id, bot, description)

def main():
    print("=" * 75)
    print("      IT AUTOMATION INCIDENT TICKET MANAGER")
    print("=" * 75)
    load_sample_data()
    print("\n10 sample incident tickets have been loaded for testing.")

    while True:
        print("\n----- MENU -----")
        print("1. Add Incident Ticket")
        print("2. Display All Active Incident Tickets")
        print("3. Search Incident Ticket")
        print("4. Remove Resolved Incident Ticket")
        print("5. Count Active Incident Tickets")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ").strip()

        if choice == "1":
            incident_id = input("Enter Incident ID: ").strip()
            bot = input("Enter Bot Name: ").strip()
            description = input("Enter Short Description: ").strip()
            add_ticket(incident_id, bot, description)

        elif choice == "2":
            display_tickets()

        elif choice == "3":
            incident_id = input("Enter Incident ID to search: ").strip()
            search_ticket(incident_id)

        elif choice == "4":
            incident_id = input("Enter Incident ID to remove: ").strip()
            remove_ticket(incident_id)

        elif choice == "5":
            count_active_tickets()

        elif choice == "6":
            print("\nExiting program. Goodbye!")
            break

        else:
            print("\nInvalid choice. Please try again.")

```

```
if __name__ == "__main__":
    main()
```

VII. Program Output

Below are the program outputs demonstrating each required operation, tested using the 10 sample incident tickets.

The screenshot shows the Visual Studio Code editor with the file `LAB_EXER1.py` open. The code defines a program menu and a function to add tickets. The terminal output shows the program has loaded 10 sample incident tickets and displays the following menu:

```

Ticket INC1392948 added successfully.
10 sample incident tickets have been loaded for testing.

----- MENU -----
1. Add Incident Ticket
2. Display All Active Incident Tickets
3. Search Incident Ticket
4. Remove Resolved Incident Ticket
5. Count Active Incident Tickets
6. Exit
Enter your choice (1-6): 2

```

Figure 1. Program menu after loading the 10 sample incident tickets.

The screenshot shows the Visual Studio Code editor with the file `LAB_EXER1.py` open. The terminal output shows the program has loaded 10 sample incident tickets and displays the following menu:

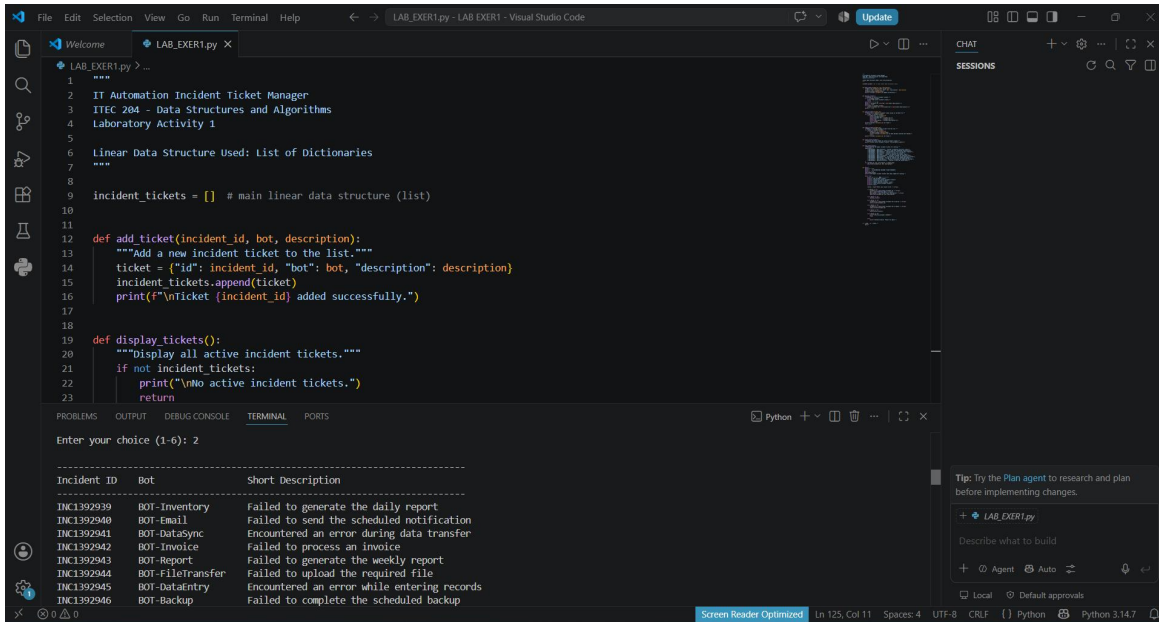
```

----- MENU -----
1. Add Incident Ticket
2. Display All Active Incident Tickets
3. Search Incident Ticket
4. Remove Resolved Incident Ticket
5. Count Active Incident Tickets
6. Exit
Enter your choice (1-6): 1
Enter Incident ID: INC1392999
Enter Bot Name: Bot-Test
Enter Short Description: Sample new ticket for testing

Ticket INC1392999 added successfully.

```

Figure 2. Add Incident Ticket operation.

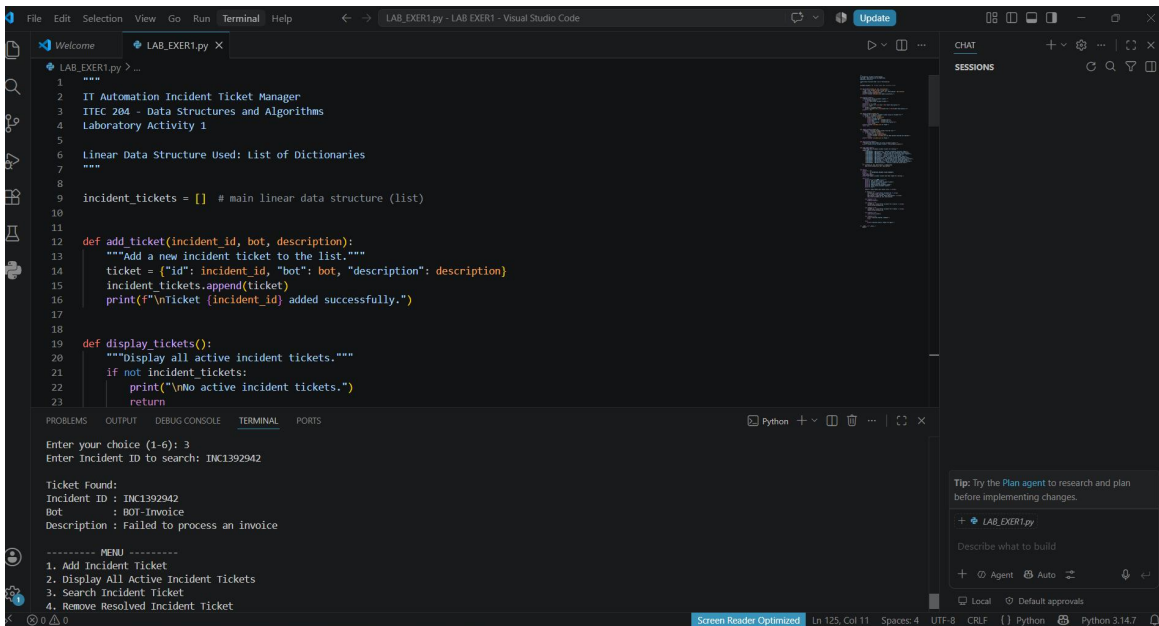


```
1 """
2 IT Automation Incident Ticket Manager
3 ITEC 204 - Data Structures and Algorithms
4 Laboratory Activity 1
5
6 Linear Data Structure Used: List of Dictionaries
7 """
8
9 incident_tickets = [] # main linear data structure (list)
10
11
12 def add_ticket(incident_id, bot, description):
13     """Add a new incident ticket to the list."""
14     ticket = {"id": incident_id, "bot": bot, "description": description}
15     incident_tickets.append(ticket)
16     print(f"\nTicket {incident_id} added successfully.")
17
18
19 def display_tickets():
20     """Display all active incident tickets."""
21     if not incident_tickets:
22         print("\nNo active incident tickets.")
23         return
```

Enter your choice (1-6): 2

Incident ID	Bot	Short Description
INC1392939	BOT-Inventory	Failed to generate the daily report
INC1392940	BOT-Email	Failed to send the scheduled notification
INC1392941	BOT-Datasync	Encountered an error during data transfer
INC1392942	BOT-Invoice	Failed to process an invoice
INC1392943	BOT-Report	Failed to generate the weekly report
INC1392944	BOT-Filetransfer	Failed to upload the required file
INC1392945	BOT-Dataentry	Encountered an error while entering records
INC1392946	BOT-Backup	Failed to complete the scheduled backup

Figure 3. Display All Active Incident Tickets operation.



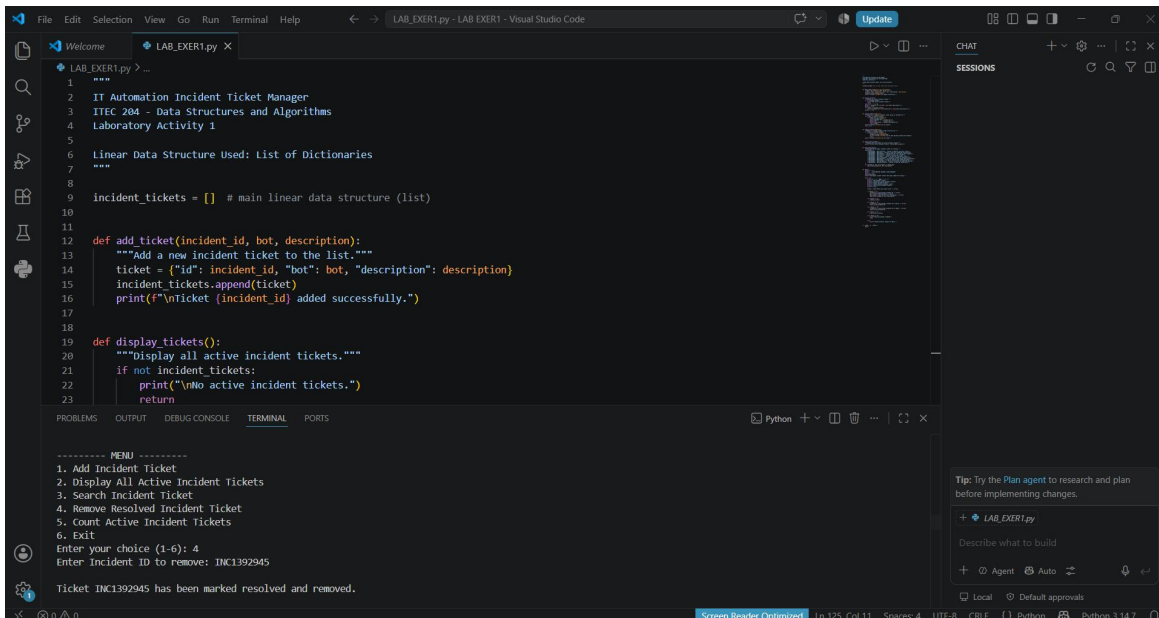
```
1 """
2 IT Automation Incident Ticket Manager
3 ITEC 204 - Data Structures and Algorithms
4 Laboratory Activity 1
5
6 Linear Data Structure Used: List of Dictionaries
7 """
8
9 incident_tickets = [] # main linear data structure (list)
10
11
12 def add_ticket(incident_id, bot, description):
13     """Add a new incident ticket to the list."""
14     ticket = {"id": incident_id, "bot": bot, "description": description}
15     incident_tickets.append(ticket)
16     print(f"\nTicket {incident_id} added successfully.")
17
18
19 def display_tickets():
20     """Display all active incident tickets."""
21     if not incident_tickets:
22         print("\nNo active incident tickets.")
23         return
```

Enter your choice (1-6): 3
Enter Incident ID to search: INC1392942

Ticket Found:
Incident ID : INC1392942
Bot : BOT-Invoice
Description : Failed to process an invoice

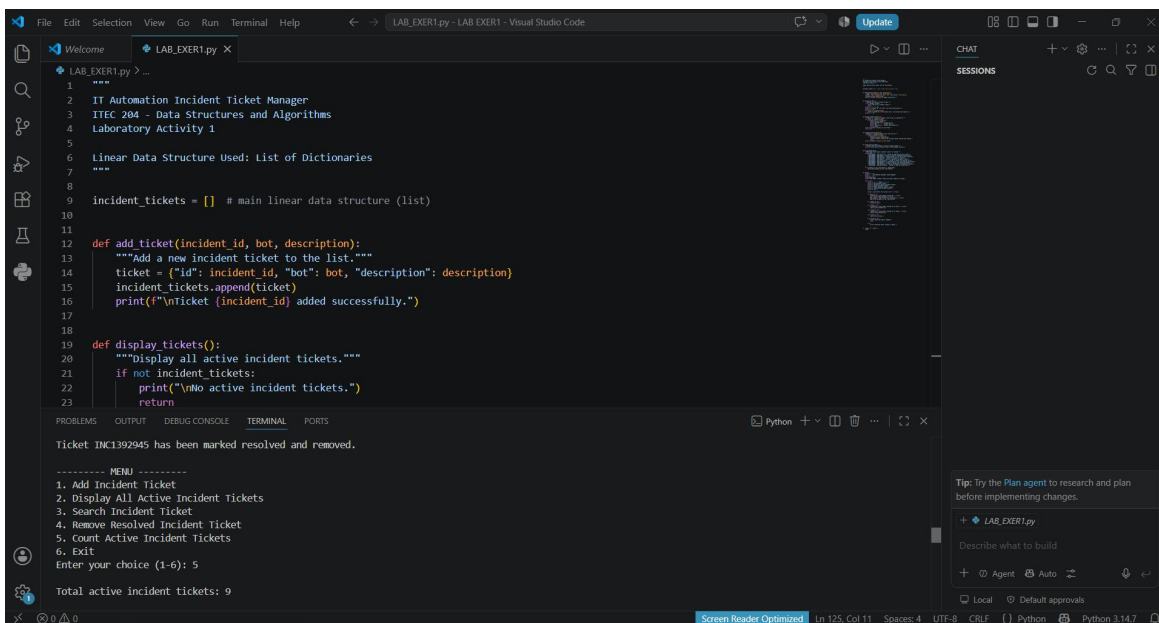
----- MENU -----
1. Add Incident Ticket
2. Display All Active Incident Tickets
3. Search Incident Ticket
4. Remove Resolved Incident Ticket

Figure 4. Search Incident Ticket operation.



```
1 """
2 IT Automation Incident Ticket Manager
3 ITEC 204 - Data Structures and Algorithms
4 Laboratory Activity 1
5
6 Linear Data Structure Used: List of Dictionaries
7 """
8
9 incident_tickets = [] # main linear data structure (list)
10
11
12 def add_ticket(incident_id, bot, description):
13     """Add a new incident ticket to the list."""
14     ticket = {"id": incident_id, "bot": bot, "description": description}
15     incident_tickets.append(ticket)
16     print(f"\nTicket {incident_id} added successfully.")
17
18
19 def display_tickets():
20     """Display all active incident tickets."""
21     if not incident_tickets:
22         print("\nNo active incident tickets.")
23     return
24
25
26 ----- MENU -----
27 1. Add Incident Ticket
28 2. Display All Active Incident Tickets
29 3. Search Incident Ticket
30 4. Remove Resolved Incident Ticket
31 5. Count Active Incident Tickets
32 6. Exit
33 Enter your choice (1-6): 4
34 Enter Incident ID to remove: INC1392945
35
36 Ticket INC1392945 has been marked resolved and removed.
```

Figure 5. Remove Resolved Incident Ticket operation.



```
1 """
2 IT Automation Incident Ticket Manager
3 ITEC 204 - Data Structures and Algorithms
4 Laboratory Activity 1
5
6 Linear Data Structure Used: List of Dictionaries
7 """
8
9 incident_tickets = [] # main linear data structure (list)
10
11
12 def add_ticket(incident_id, bot, description):
13     """Add a new incident ticket to the list."""
14     ticket = {"id": incident_id, "bot": bot, "description": description}
15     incident_tickets.append(ticket)
16     print(f"\nTicket {incident_id} added successfully.")
17
18
19 def display_tickets():
20     """Display all active incident tickets."""
21     if not incident_tickets:
22         print("\nNo active incident tickets.")
23     return
24
25
26 ----- MENU -----
27 1. Add Incident Ticket
28 2. Display All Active Incident Tickets
29 3. Search Incident Ticket
30 4. Remove Resolved Incident Ticket
31 5. Count Active Incident Tickets
32 6. Exit
33 Enter your choice (1-6): 5
34
35 Total active incident tickets: 9
```

Figure 6. Count Active Incident Tickets operation.

VIII. Conclusion

Through this activity, I learned that a linear data structure such as a list is an efficient way to organize records that need to be added, searched, and removed individually, since it allows straightforward sequential access to each element. I was also able to apply core list operations - `append()`, `remove()`, iteration, and `len()` - to a practical, real-world scenario involving the management of incident tickets generated by automation bots. This exercise strengthened my understanding of how linear data structures serve as the foundation for building simple, functional record-management systems.